

Scene Graph Generation for Indoor Scene Understanding

Sanzida Akter Selmi, Tanjimul Islam Tamim, Soptorshi Sarkar Smriti

Department of Computer Science & Engineering

East West University, Dhaka, Bangladesh

Student IDs: 2023-3-60-441, 2023-3-60-448, 2023-3-60-447

Course: CSE366 – Artificial Intelligence, Semester: Summer 2026, Section: 05, Group No: 10

Submitted to: Md Al-Imran, Senior Lecturer

Abstract—This project report documents the development and implementation of a Scene Graph Generation system for Indoor Scene Understanding with integrated semantic reasoning and natural language query-answering capabilities. The work extends upon a baseline scene graph generation pipeline by introducing a semantic reasoning module that ensures relational consistency and derives proximity relationships, coupled with a natural language query interface that enables intuitive human-computer interaction. The system maintains transparency and explainability through rule-based reasoning rather than learned models, making results easily traceable and debuggable.

I. INTRODUCTION

Scene understanding is a thing that Artificial Intelligence systems need to work properly in real-world indoor environments. This includes service robots, smart-home systems and intelligent virtual assistants. They all need scene understanding to work. The old way of doing scene understanding is limited to finding objects. It can tell us what objects are in a scene. It does not tell us how those objects are related to each other in space or in meaning. For example if we find a chair a table and a cup the old way does not tell us if the cup is on the table if the chair is near the table or if the path to the door is blocked.

This project solves that problem by using a scene graph. A scene graph is a way of representing a scene. The objects we find become points. The relationships between them become lines with labels. This turns a list of objects into a model of the scene that we can ask questions about. This is important for any system that needs to think about its environment not see it.

The project was done in three parts:

Milestone 1: We looked at what other people have done with scene graphs found the gap in research and decided on the direction of the project.

Milestone 2: We made a version of a scene-graph generation method. This included finding objects figuring out how they are related in space and building and visualizing the graph.

Milestone 3: Implements the final project idea-extending the Milestone 2 graph with a semantic reasoning module and a natural-language query-answering interface, directly targeting the gap identified in Milestone 1.

II. PROJECT OBJECTIVE

The goal of the project is to help AI systems understand rooms inside buildings by (a) finding items, (b) showing how these items are connected in a picture made of lines and nodes and (c) letting the system think and answer questions about the picture not just draw a fixed picture. This ability to think and answer questions is what makes a picture of the room for example a robot asking “is there something on the table?” before putting something there or a home device telling “where are my keys?” based on the last known positions, in the picture.

III. SEMANTIC REASONING AND QUERY ANSWERING MODULE

This stage uses the baseline pipeline, which includes detection and geometric relationship inference without making any changes to it. It then adds two components to it: a semantic reasoning module and a natural-language query-answering module. There is also an important change to how objects are named in this stage.

A. Unique Node Naming

In the baseline all objects of the class had the same label. For example every chair in the image was called “chair”. This made it impossible to tell one chair from another when there were chairs in the image. In this stage detected objects are given names like chair_1, chair_2 and table. If there is one table it is just called “table”. This is necessary for the semantic reasoning and query answering to work. Without it would be impossible to answer a question like “Is the chair on the left near the table?” because both chairs would have the same name.

B. Semantic Reasoning Module

The semantic reasoning module takes the information produced by the baseline relationship-inference step. Adds two new types of information:

Inverse-relation consistency: For every relationship that the geometric module detects there is a relationship. For example “on” has a relationship “under” and “left of” has an opposite relationship “right of”. If the geometric module

detects “cup on table” but not “table under cup” the reasoning module adds the missing relationship automatically. This ensures that the information is consistent and accurate.

Derived “near” relation: If two objects are connected by any relationship, such as “on” “under” “left of” “right of” “in front of” or “behind” the reasoning module also adds a “relationship between them. This is an addition that helps answer questions like “what is near the sofa?” that the geometric module cannot answer on its own.

The semantic reasoning module uses rules to make these additions than learned models. This is a design choice because it makes the module transparent and easy to understand. Every addition can be traced back to the rule that made it which’s helpful for debugging and explaining the answers to the user.

C. Natural-Language Query Answering

The `answer_query` function allows users to ask questions about the scene in language. It supports types of questions including:

What is on/under/near an object? For example “what is on table?” lists all objects that’re on the table.

Is an object near another object? For example “is chair near table?” answers yes or no.

Is an object in a relationship with another object? For example “is chair left of table?” answers yes or no.

List all relationships for an object. For example “list relations chair” lists all relationships that involve the chair.

The function is case-insensitive. Can handle questions without the numeric suffix. For example typing “chair” will automatically resolve to “chair_1” if there is one chair in the scene. The function also has a loop that allows users to ask multiple questions about the same scene without re-running the pipeline.

D. Visualization

The final scene graph is drawn with two types of edges: black arrows for relationships that come from the geometric detector and dashed orange arrows for relationships that are added by the semantic reasoning module. This makes it easy to see the contributions of the reasoning module in the graph itself. The Semantic Reasoning and Query Answering Module is important for understanding the scene and answering questions, about it. The semantic reasoning. The natural-language query-answering module work together to provide a clear and accurate picture of the scene.

IV. RESULTS

A. Input Image



Fig. 1. Input Image

B. Object Detection Output

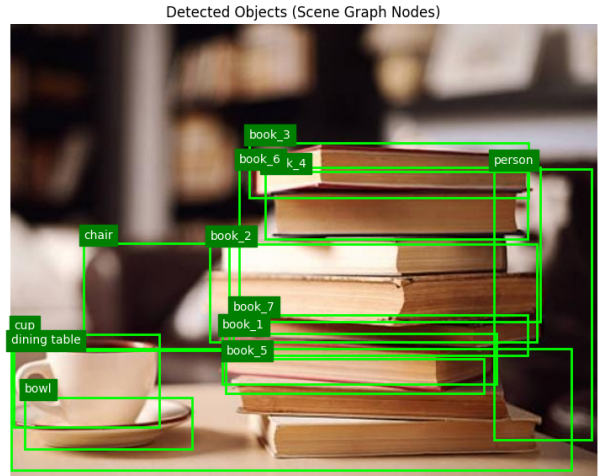


Fig. 2. Object Detection Output

C. Scene Graph Triplets (Detected)

```
Scene graph triplets (subject - predicate - object):
cup --right of--> book_1
cup --right of--> book_2
cup --right of--> book_3
cup --right of--> book_4
cup --right of--> book_5
cup --right of--> book_6
cup --right of--> book_7
cup --in front of--> bowl
cup --behind--> chair
cup --right of--> dining table
book_1 --left of--> cup
book_1 --behind--> book_2
book_1 --behind--> book_3
book_1 --behind--> book_4
book_1 --on--> book_5
book_1 --behind--> book_6
book_1 --right of--> person
book_1 --behind--> book_7
book_1 --left of--> bowl
book_1 --left of--> chair
book_1 --left of--> dining table
book_2 --left of--> cup
book_2 --in front of--> book_1
book_2 --behind--> book_3
book_2 --behind--> book_4
book_2 --in front of--> book_5
book_2 --under--> book_6
book_2 --right of--> person
book_2 --in front of--> book_7
book_2 --left of--> bowl
book_2 --left of--> chair
book_2 --in front of--> dining table
book_3 --left of--> cup
```

Fig. 3. Scene Graph Triplets (Detected)

D. Reasoning Module Output

```
... Original triplets: 124
After adding inverse relations: 124
After adding derived near relations: 248

New facts added by the reasoning module:
book_4 --near--> chair
book_3 --near--> person
book_5 --near--> person
book_1 --near--> book_4
book_1 --near--> bowl
book_4 --near--> person
book_2 --near--> book_6
book_2 --near--> book_3
book_1 --near--> book_2
book_5 --near--> book_6
book_6 --near--> chair
book_7 --near--> bowl
book_3 --near--> book_6
book_2 --near--> cup
book_4 --near--> book_6
book_6 --near--> person
book_5 --near--> cup
book_1 --near--> dining table
book_2 --near--> book_7
book_3 --near--> cup
book_5 --near--> book_7
bowl --near--> dining table
```

Fig. 4. Reasoning Module Output

E. Sample Queries and Answers

```
Ask about the scene (or "exit"): what is on book
A: book_6 is on book_2; book_1 is on book_5; book_4 is on book_6
Ask about the scene (or "exit"): what is near cup
A: book_2, book_5, book_3, book_4, book_6, book_1, bowl, book_7, chair, dining table are near cup
Ask about the scene (or "exit"): exit
```

Fig. 5. Sample Queries and Answers

F. Reasoning-Augmented Scene Graph

Scene Graph with Reasoning-Derived Edges (dashed orange = inferred by reasoning module)



Fig. 6. Reasoning-Augmented Scene Graph

V. INFERENCE AND DISCUSSION

The results show that a simple and easy to understand rule-based reasoning layer can really help a geometric scene graph without needing any training data or a special reasoning model. By making sure the relationships are consistent the system creates a graph that makes sense no matter what direction the geometric rule was applied. This was a problem with the output that was not clear before. By figuring out a way to describe things that are near each other the system can answer questions about how close things are to each other which the original detector was not designed to do.

The query-answering interface shows that this detailed and consistent graph can answer the kinds of questions that we wanted to answer in the first place. Like the kinds of questions a robot might ask, such as “is there anything on the table before I put this down?” or a smart-home assistant might ask, such as “what is near the sofa?”. Because every answer is based on an understandable rule the systems answers are also easy to explain: for every answer we can see exactly which detection and which rule led to it.

At the time because the system is based on rules rather than learning, it cannot understand relationships that need knowledge of the world what objects can be used for or small visual details. Like “holding” “wearing” or “riding”. The way a special classifier trained on a dataset like Visual Genome can. The system also cannot figure out what is going on in blocked scenes where simple geometric rules are not enough. The

query interface can only understand a fixed set of questions using regular-expression matching than really understanding natural language so it cannot answer questions that are phrased in any way. Scene graph and geometric scene graph are really important for the system to work properly and to answer questions, about the scene.

VI. LIMITATIONS

The object vocabulary that we are working with is limited to the 80 categories that are found in COCO. This is because the detector was pretrained on COCO. It was not fine-tuned on Visual Genome or a dataset that is specific to scenes.

We only look at the geometry of objects to figure out their relationships. We see if the boxes that surround the objects overlap and where they are in relation to each other. We do not have a way to understand complex relationships like “holding” or “wearing” or “riding”. The detector is not able to learn these things on its own.

When we answer questions we look for patterns in the words that are used. We have a set of templates that we know how to answer. If the question does not fit one of these templates we will not be able to understand it.

VII. CONCLUSION

This project did what we first thought of when we looked at the literature. We wanted to make scene graph generation better by adding reasoning and interactive query answering. We started with the pipeline and added a module that uses rules for semantic reasoning and a way to answer questions in natural language. This fixed the problem we found in the literature. Other scene graph work like Xu et al. 2017 Wald et al. 2020. Visual Genome only focused on making the graph not on understanding it or answering questions about it. Our system takes a picture of a room finds the objects makes a scene graph adds information to it and answers questions about the scene. The system scene graph generation is transparent, explainable and reproducible without needing a lot of data to train it. This project gives us a starting point for understanding scenes, which is useful, for robots, smart homes and intelligent assistants just like we said in our original proposal. The scene graph generation is a part of this project and it helps us to understand the scene better.

REFERENCES

- [1] Xu D, Zhu Y, Choy CB, Fei-Fei L (2017). *Scene graph generation by iterative message passing*. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.
- [2] Wald T, Dhamo H, Navab N, Tombari F (2020). *Learning 3D semantic segmentation of large-scale indoor scenes*. In: 2020 International Conference on 3D Vision (3DV), IEEE.
- [3] Krishna R, Zhu Y, Groth O, Johnson J, Hata K, Kravitz J, Chen S, Kalantidis Y, Li LJ, Shamma DA, Fei-Fei L, Koller D (2017). *Visual Genome: Connecting Language and Vision Using Crowdsourced Dense Image Annotations*. International Journal of Computer Vision.